Appointment Booking & Capacity Planning System

Microservices Architecture with CI/CD on AWS (ECR + ECS)

Module	Architecting Scalable Systems
Institution	National University of Singapore (NUS)
Team Size	3
Version	v1.0
Date	28 Jan 2026

Prepared by: *Brendan Ng, Jun Han, Phillip*

1. Executive Summary

This proposal outlines the design and implementation of a cloud-native Appointment Booking & Capacity Planning System for facilities such as clinics, consultation rooms, sports courts, and shared resources. The solution is built using a microservices architecture and deployed on AWS using Elastic Container Registry (ECR) and Elastic Container Service (ECS). A key technical focus is correctness under contention: the system prevents double-booking through transactional controls and automated regression testing integrated into the CI/CD pipeline.

2. Background and Problem Statement

Many booking solutions work well at small scale but struggle during peak demand. Common failure modes include double-booking, stale availability displays, inconsistent cancellations/reschedules, and delayed reminders. These issues become more severe when multiple facilities share limited capacity and when concurrent users attempt to reserve the same timeslot.

3. Project Objectives

3.1 Functional Objectives

- Allow users to search availability and book time slots for a selected resource.
- Support cancellation and rescheduling while preserving capacity rules.
- Send booking confirmations and automated reminders (e.g., 24 hours and 2 hours prior).
- Provide administrative functions to manage resources, operating hours, blackout dates, and slot capacity.

3.2 Engineering Objectives

- Demonstrate microservices decomposition with independent deployment and scaling on ECS.
- Guarantee booking correctness under concurrent requests (no double-booking).
- Implement CI/CD (GitHub Actions -> ECR -> ECS) with environment promotion (Dev -> UAT).
- Implement observability (structured logs, metrics, tracing) to support troubleshooting and operations.

4. Scope

4.1 In Scope

- Three core microservices: Resource Service, Booking Service, Notification Service.
- API-first implementation with OpenAPI/Swagger documentation; optional minimal web UI for demo.
- Automated tests (unit, integration, concurrency regression) and post-deploy smoke tests.
- Deployment on AWS ECS with container images stored in ECR.
- Operational visibility via logging, metrics, and tracing.

4.2 Out of Scope

- Payment processing and billing.
- Complex identity proofing (cookies).

5. Proposed Solution Overview

The system is organized into three independently deployable services. Each service owns its domain responsibilities and data. Service boundaries are designed to reduce coupling and allow independent scaling based on load patterns.

Service	Responsibilities
Resource Service	Manage resources (facilities), operating hours, slot duration, blackout dates, and capacity rules.
Booking Service	Create/cancel/reschedule bookings, enforce capacity constraints, prevent double-booking under concurrency.
Notification Service	Send confirmations and reminders; process booking events asynchronously to avoid blocking bookings.

6. Architecture and Deployment

The solution is deployed on AWS using containerized microservices. Traffic is routed through an Application Load Balancer (ALB) to ECS services.

The following diagram summarizes the proposed simple architecture:

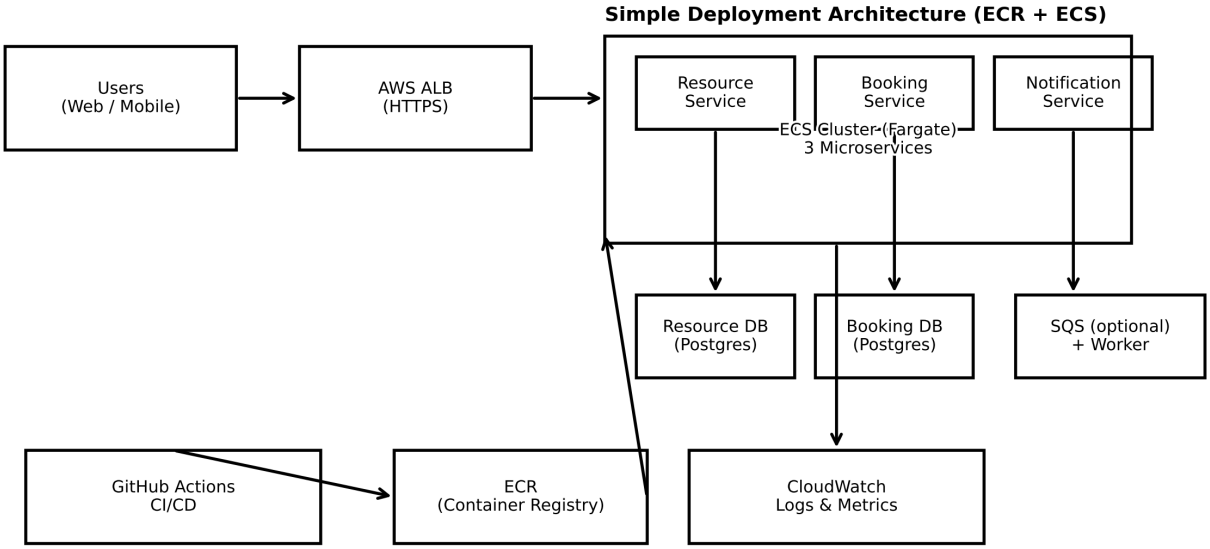

Figure 1: Simple deployment architecture on AWS (ECR + ECS).

6.1 Key AWS Components

- Application Load Balancer (ALB): routes HTTPS traffic to ECS services. (3 domain name required depending on the cost we might omit https)
- ECS (Fargate or EC2 launch type): runs containerized microservices with rolling deployments.

- ECR: stores versioned container images tagged by git commit SHA and release version.
- RDS PostgreSQL: transactional persistence for booking and resource data (separate schemas/databases per service).
- SQS (optional): decouples notification delivery from the booking transaction path.
- CloudWatch: centralized logs and service-level metrics.

7. Data Strategy and Consistency

7.1 Database Ownership

Each microservice owns its database schema to preserve service autonomy and reduce cross-service coupling. The project will use PostgreSQL for transactional correctness; service data separation can be implemented as separate databases or separate schemas with strict access controls.

7.2 Preventing Double-Booking (Core Correctness Requirement)

The Booking Service enforces capacity constraints using database transactions and row-level locking. A capacity ledger is maintained per resource timeslot, and booking requests execute a single transaction that locks the slot row, verifies remaining capacity, writes the booking record, and updates reserved counts. This approach prevents race conditions and ensures that bookings never exceed capacity even under concurrent requests.

- Implementation approach (transactional locking):
    - Maintain slot capacity records keyed by (resource_id, slot_start).
    - Use SELECT ... FOR UPDATE to lock the relevant slot row during booking.
    - Reject requests when reserved_count equals capacity.
    - Record idempotency keys to guard against duplicate client retries (optional stretch goal).

8. Non-Functional Requirements

8.1 Scalability

- Stateless services horizontally scale on ECS; scaling policies can be based on CPU, memory, or request rates.
- Booking Service focuses on high-contention slot updates using efficient locking strategies.
- Notifications run asynchronously to reduce load on the critical booking path.

8.2 Availability and Resilience

- ECS health checks and auto-restart for failed tasks.
- Retry policies for notification delivery; optional dead-letter queue for failed messages.
- Graceful degradation: booking remains available even if reminder processing is delayed.

8.3 Security

- HTTPS termination at ALB; JWT-based authentication for API access.
- Role separation for administrators vs standard users.

- Secrets stored in AWS Secrets Manager or SSM Parameter Store; least-privilege IAM for ECS tasks.

8.4 Observability

- Structured JSON logging with correlation IDs across services.
- Key metrics: booking success rate, conflict rate, latency p95, reminder success/failure counts.
- Distributed tracing via OpenTelemetry instrumentation.

9. CI/CD Approach

9.1 Workflow and Environments

The project uses GitHub Actions to build, test, and deploy container images. Pull requests run quality gates (lint, unit tests, integration tests). Merges to the main branch trigger image builds, pushes to ECR, and automated deployment to the Dev environment. Promotion to UAT is controlled by a manual approval gate using GitHub Environments.

9.2 Pipeline Stages

- PR checks: lint, unit tests, build, integration tests (docker compose), concurrency regression tests for double-booking.
- Main deploy: build and tag images, push to ECR, update ECS task definitions, rolling deployment, post-deploy smoke tests.
- Promotion: deploy the same immutable image tag to UAT after approval; repeat smoke tests.

10. Testing Strategy

- Unit tests: business rules, input validation, availability calculations.
- Integration tests: database interactions, transaction behavior, API contracts.
- Concurrency regression tests: parallel booking attempts on the same slot; validate capacity is never exceeded.
- Smoke tests: health checks and an end-to-end booking flow after each deployment.

11. Proposed Project Plan and Milestones

Timeline	Milestone
Week 1-2	Finalize requirements, API contracts, and infrastructure baseline (ECS/ECR, networking).
Week 3-4	Implement Resource Service and Booking Service skeleton, DB schemas, basic booking flow.
Week 5	Implement double-booking prevention and concurrency regression tests.
Week 6	Implement Notification Service and reminders; integrate async workflow (SQS optional).
Week 7	Add observability, hardening, and resilience

	improvements.
Week 8	Finalize CI/CD, UAT promotion, demo scripts, and documentation.

12. Team Roles and Responsibilities

- Member A (Platform/DevOps): ECS/ECR setup, networking/IAM, CI/CD pipelines, environment promotion, monitoring/logging.
- Member B (Booking Lead): booking lifecycle APIs, concurrency control, database transactions, regression tests.
- Member C (Resource + Notification Lead): resource rules and availability, notification workflows, reminder scheduling, demo UI/API docs.

13. Deliverables

- Deployed system on AWS ECS with Dev and UAT environments.
- Source repository with documented microservices and API specifications.
- CI/CD pipelines with automated gates and promotion workflow.
- Test evidence demonstrating no double-booking under concurrent requests.
- Architecture documentation and operational runbook for demo.

14. Risks and Mitigations

Risk	Mitigation
Feature scope grows beyond schedule.	Keep UI minimal; prioritize correctness and CI/CD deliverables; timebox optional features.
Concurrency logic is implemented late.	Implement locking/constraints early; add concurrency tests from Week 3 onward.
Notification failures complicate demo.	Use queue-based processing with retries; allow eventual consistency for reminders.

15. Success Criteria

- Correctness: capacity is never exceeded; double-booking is prevented under parallel requests.
- Deployability: automated Dev deployments and controlled promotions to UAT using immutable image tags.
- Operability: logs, metrics, and traces support root-cause analysis of simulated failures. Expandable
- Scalability: services can be scaled independently; booking throughput remains stable in basic load tests.